

Real-world (a cold design-review interview on your finished capstone) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `capstone_review.md` : the final exam of the curriculum: a cold review of your finished capstone, jumping between cost, testing, database and auth at a reviewer's pace. **If not:** the first time you faced this kind of cross-examination would be a final-round interview.

Write your own version first, in `practice/capstone_review.md` , then compare. Read both depths — they're 2 real levels of the same answer.

Model interview for: "Interview me about my capstone" — this solution uses the same ContentForge / Project 5 system from Docs 16 and 17 as the running example, extended to the full production system (RAG, database, auth, deployment, cost) this document actually asks you to build. (In this example, the capstone design added a fact-checker agent to Project 4's team, to check each draft's claims against the RAG sources.)

Basic Version

Approach 1 — the plain version

Story: answer whatever is asked, in whatever order, and say plainly when a choice was a default. **If not:** you'd invent a reason for every number, and a reviewer would catch the first invented one.

Reviewer: "Walk me through it, then I'll ask about whatever I want."

"ContentForge is a multi-agent system that researches, drafts, fact-checks, and publishes content, with a supervisor deciding what happens next. For this capstone it's the full production version — it has a database, an API, login, RAG over a source-document store, and it's deployed with monitoring and a cost cap."

Reviewer, jumping straight to database, no warm-up: "Why two databases instead of one?"

"I used Postgres for structured data — users, jobs, billing — and a separate vector store for the RAG chunks. Postgres gives me transactional guarantees for things like billing that a vector store doesn't provide, and the vector store is actually good at similarity search, which Postgres alone isn't at this scale."

Reviewer, jumping to auth: "How many roles does your permission system have, and why that number?"

"Three — admin, editor, and viewer. Honestly, that was closer to a reasonable default than something the requirements specifically asked for."

This is a complete, honest Basic pass — it answers whatever's asked, in whatever order it's asked, and is willing to say plainly when a choice was a default rather than invent a reason for the exact number 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — specific and tied to requirements

Story: every answer tied to a specific requirement or failure mode, including the gaps you volunteer.

If not: your answers would describe the pieces without saying why each one is there.

Reviewer: "Same as before, but I'm not going in system order — I'll jump around."

Reviewer, opening on cost, not architecture: "Your requirements said \$200/month. Walk me through what's actually keeping you under that."

"Three things enforce it: a per-request token budget on the generation calls, the retrieval cache in front of the vector store that avoids re-embedding repeated queries, and a monthly spend alert wired to the LLM provider's usage API that pages me before the account gets close to the cap. The cache is the biggest lever — most of the cost at this traffic volume is repeated similar queries, not unique ones."

Reviewer, jumping to testing: "What's actually tested versus what you're hoping works?"

"Each agent's node function has unit tests against fixed inputs — the fact-checker's grounding logic, specifically, is tested against cases where the source material does and doesn't support a claim, since that's the highest-stakes piece. The supervisor's routing logic has integration tests that run a full simulated request through 2-3 known paths, including the backward loop from fact-check failure to the writer. What's NOT tested end-to-end: the actual deployed API under real concurrent load — I've load-tested it manually once, not as part of CI, which is a real gap for a system with a stated 50,000 requests/day target."

Reviewer, back to database, from a different angle than before: "You said Postgres and the vector store are separate because of different access patterns. What happens to a request if the vector store is briefly unavailable but Postgres is fine?"

"That's handled as a degraded-mode path, not a full failure — the agent's RAG tool call is wrapped the way Doc14 trains, so a vector-store timeout doesn't crash the request. It falls back to answering from the model's general knowledge with an explicit disclaimer in the output that retrieval was unavailable, rather than either crashing or silently answering as if grounding had succeeded."

Why this level works: each answer is specific to the actual requirement or actual failure mode, not a generic description of the piece. The testing answer volunteers the real gap (no automated load test) unprompted, which is what Doc17's material already established as a stronger signal than

waiting to be caught. The database-availability answer shows the design was thought through past the happy path, tying back to Doc14's failure-handling vocabulary.

Approach 2 — when the pushback comes back harder

Story: a strong reviewer connects two of your own answers and asks which one is true. This practices admitting a real contradiction, naming which answer holds, and giving a concrete fix. **If not:** you'd defend both answers with reasoning invented on the spot.

Reviewer, pushing on the auth answer from the Basic pass: "You told me 3 roles was closer to a default than a real requirement. If I push on that right now — defend it anyway, or tell me what you'd actually check."

Model response: "I won't manufacture a justification for exactly 3 — that would be the same rationalization trap the design_self_review exercise is built to catch: writing a requirement after the fact to match a number I already picked. What I'd actually do is go back to the live requirements handoff and check whether it named specific roles or access levels at all. If it didn't, 3 stays a reasonable starting default, but I'd say so explicitly rather than defend it as principled. If it did name something more specific — say, a distinction between 'can publish' and 'can edit but not publish' that my 3 roles collapse into one — then my design has a real gap, not just an unjustified default, and I'd fix the role model before calling this done."

Reviewer, now finding a seam between two separate answers given minutes apart: "Earlier you said the vector-store fallback answers from general knowledge with a disclaimer when retrieval is unavailable. But you also told me your fact-checker's whole job is verifying claims against retrieved sources, and that ungrounded claims get treated as unsure and escalated. If retrieval is down, the agent is now generating from general knowledge on purpose — doesn't your own fact-checker have to flag every single one of those responses as ungrounded and escalate them all, which would make your 'graceful degradation' actually mean 'escalate everything until retrieval comes back'? Which of your two answers is actually true?"

Model response: "That's a real inconsistency, not something I'd noticed until you connected them just now. As designed, yes — the fact-checker's grounding rule and the retrieval-fallback path are in direct tension: the fallback intentionally produces an ungrounded answer, and the fact-checker's job is to catch exactly that and escalate it. Right now the honest answer is that during a retrieval outage, the system would likely escalate most or all requests, which is a much more disruptive failure mode than 'gracefully degraded with a disclaimer' made it sound like a minute ago. That's not acceptable as a real production behavior at the stated scale, and I'd rather say so than paper over it. The fix I'd actually make: during a detected retrieval outage, the fact-checker's threshold needs a distinct outage-aware mode — either it stops requiring grounding for the disclaimer-flagged responses specifically (treating the disclaimer itself as the honesty mechanism, not full escalation), or the system stops accepting new requests during the outage entirely rather than serving degraded answers that then get expensively escalated one by one. I hadn't reconciled these two pieces of the

design against each other before this conversation, and that's a genuine gap I'm finding right now, not one I'm defending."

Reviewer, one more, testing whether the "genuine vs. default" instinct generalizes: "Give me one more piece of your design you now suspect was a default dressed up as a decision, without me asking about it specifically."

Model response: "The Docker Compose deployment running Postgres and the vector store as one restart unit. I told myself that was fine because the stated scale doesn't need independent scaling yet — and that part's true. But the actual reason I set it up that way was that it was the simplest thing to get running first, and I only found the scale-based justification afterward. It happens to be a defensible justification, but I should be honest that I found the reason after the fact, not before — which is exactly the bias check from the design_self_review exercise, just running on deployment instead of on a written requirements table."

Why this level works: the auth answer refuses to invent a justification under direct pressure and instead names the specific check that would resolve it — which is the same discipline as this document's own design_self_review exercise, applied live instead of on paper. The seam-finding exchange is the hardest realistic move in the whole document: admitting, in real time, that two of your own answers contradict each other, naming which one is more true, and proposing a specific fix instead of defending both as if they'd always been compatible. The final unprompted admission shows the "genuine decision vs. default" instinct has actually generalized, not just been rehearsed for the one row prepared in advance.

What a weaker answer misses: on the auth pushback, a weaker answer either invents a retroactive justification for exactly 3 roles (the rationalization Hint 1 warns about) or collapses entirely ("you're right, it's wrong") without proposing what to actually check. On the seam-finding exchange, a weaker answer either fails to notice the two answers conflict at all, or notices but defends both as compatible with strained reasoning invented on the spot — both read as less trustworthy than a plain "you're right, here's the actual fix."

Which depth should you actually deliver, and when? Open every answer at the Basic answer's plain, direct level — state what you built and why, without over-explaining. Let it rise into Approach 1's specificity as the reviewer asks follow-ups. Don't pre-load Approach 2's contradictions or your own weakest points before anyone asks — but the moment a reviewer finds a real seam between two of your decisions, or pushes on something you already flagged as a default, that's exactly the cue to do what this document's whole final stage is testing: hold your ground where you actually should, and concede plainly, with a specific fix, where you actually shouldn't. That's the entire difference between defending a capstone and having genuinely built one.